



Bahria University Islamabad

Department of Information Technology

PROJECT PROPOSAL

Low-Level Car Racing Game with Real-Time Interaction using 8086 Assembly Language

Submitted By: Faisal Amin

Enrollment No: 01-135232-019

Submitted By: Huzaifa

Enrollment No: 01-135232-058

Program: BS Information Technology

Class: BSIT 6B

Course Title: COAL

Instructor: Adnan Jelani

Submission Year: 2026

Contents

Abstract	3
1 Introduction	4
2 Problem Statement	5
3 Objectives	6
4 Project Description	7
5 Core Features	8
5.1 Gameplay Features	8
5.2 System-Level Features	8
6 Technical Design	9
6.1 Game Loop Structure	9
6.2 Data Structures	9
6.3 Key Concepts Used	9
7 Tools and Environment	11
8 Methodology	12
8.1 Phase 1: Research and Planning	12
8.2 Phase 2: Core Development	12
8.3 Phase 3: Game Mechanics	12
8.4 Phase 4: Testing and Optimization	13
9 Expected Outcomes	14
10 Innovation	15
11 Future Enhancements	16

Abstract

The purpose of this project is to design and develop a low-level car racing game using 8086 Assembly Language. The project focuses on understanding how software interacts directly with hardware resources without relying on modern programming frameworks or high-level game engines. Unlike contemporary software development approaches, where many system operations are abstracted away, this project emphasizes manual control over memory handling, keyboard interaction, screen rendering, and processor-level instructions.

The proposed system is a real-time interactive game in which the player controls a racing car moving horizontally across lanes while avoiding incoming obstacles. The game continuously processes keyboard input, updates object positions, detects collisions, refreshes the display, and calculates scores in real time. The gameplay becomes progressively difficult as the speed of incoming obstacles increases over time.

The implementation of the project will be carried out in a DOS-based environment using 8086 Assembly Language. BIOS interrupts such as INT 10h and INT 16h will be utilized for screen and keyboard operations. Direct memory manipulation techniques will be used for storing game data, tracking player positions, obstacle management, and rendering visual elements on the display.

This project aims to strengthen understanding of low-level programming concepts including CPU instruction execution, register manipulation, memory addressing, interrupt handling, and performance optimization. The final outcome will be a fully functional car racing game that demonstrates practical implementation of Assembly Language in developing interactive systems and real-time applications.

1 Introduction

Game development has evolved significantly with the introduction of high-level programming languages and advanced game engines. Modern development tools simplify many internal operations such as graphics rendering, memory management, and input handling. While these technologies improve development speed and efficiency, they often hide the low-level operations performed by the computer system.

Assembly Language provides direct interaction with hardware components and allows programmers to understand how software communicates with processors, memory, and input/output devices. Developing a game using Assembly Language offers practical knowledge of system-level programming concepts that are usually hidden in high-level environments.

This project proposes the development of a car racing game using 8086 Assembly Language. The project aims to provide practical exposure to low-level programming while implementing real-time interaction, obstacle movement, collision detection, and score tracking mechanisms.

2 Problem Statement

Most modern software applications and games are developed using high-level programming languages and frameworks that abstract hardware-level operations. As a result, students often lack practical understanding of how software directly interacts with computer architecture components such as memory, processors, interrupts, and input/output devices.

There is a need for projects that provide hands-on experience with low-level programming concepts and hardware interaction. Without this understanding, students may find it difficult to fully comprehend concepts related to operating systems, computer architecture, memory management, and processor operations.

This project addresses this issue by developing a complete interactive car racing game using 8086 Assembly Language, where all major operations including screen updates, keyboard handling, object movement, and collision detection are implemented manually using low-level programming techniques.

3 Objectives

The major objectives of this project are listed below:

- To design and develop a complete car racing game using 8086 Assembly Language.
- To understand low-level programming concepts including registers, memory addressing, and processor instructions.
- To implement real-time keyboard interaction using BIOS interrupts.
- To simulate game mechanics such as movement, collision detection, obstacle generation, and scoring.
- To improve understanding of hardware interaction and system-level operations.
- To gain practical experience with debugging and performance optimization in Assembly Language.
- To explore real-time processing techniques in low-level environments.

4 Project Description

The proposed project is a real-time car racing game developed using 8086 Assembly Language in a DOS-based environment. The player controls a car that can move horizontally across lanes using keyboard inputs. The objective of the game is to avoid collisions with incoming obstacles while surviving for the longest possible time.

The game operates using a continuous execution loop. During each cycle of the loop, the system captures user input, updates the position of the player's car, moves obstacles downward, checks for collisions, updates the score, and refreshes the display screen.

The game may use either text mode (80x25) or graphics mode (Mode 13h) depending on implementation complexity and graphical requirements. The movement of obstacles will become faster over time to increase the difficulty level and improve user engagement.

The project demonstrates practical implementation of concepts such as interrupt handling, direct memory access, register manipulation, conditional branching, looping instructions, and screen rendering.

5 Core Features

5.1 Gameplay Features

- Smooth horizontal movement of the player's car.
- Random generation of obstacles at different positions.
- Real-time collision detection between the player's car and obstacles.
- Progressive increase in difficulty through speed enhancement.
- Score calculation based on player survival time.
- Continuous game execution with responsive controls.

5.2 System-Level Features

- Keyboard handling using BIOS interrupts.
- Screen rendering using direct video memory access.
- Timing control using delay loops.
- Storage of game objects using memory variables and arrays.
- Efficient use of CPU registers for fast execution.

6 Technical Design

6.1 Game Loop Structure

The game follows a structured loop that continuously performs the following operations:

1. Capture keyboard input from the player.
2. Update the horizontal position of the player's car.
3. Generate and move obstacles downward.
4. Detect collisions between objects.
5. Update score and difficulty level.
6. Refresh and render the updated frame on the screen.

6.2 Data Structures

The project uses low-level data handling techniques for storing game information.

- Player car coordinates stored in registers or memory variables.
- Obstacle positions maintained using arrays.
- Score values stored as counters in memory.
- Speed variables used to control obstacle movement.

6.3 Key Concepts Used

- BIOS interrupts (INT 10h for display and INT 16h for keyboard input).

- Direct memory addressing techniques.
- Register manipulation for efficient execution.
- Conditional branching and loop instructions.
- Real-time processing and screen refresh operations.

7 Tools and Environment

The following tools and technologies will be used during development:

- Programming Language: 8086 Assembly Language
- Assembler: EMU8086 or MASM
- Execution Environment: DOSBox
- Hardware Platform: x86 Architecture
- Operating Environment: DOS-based system

8 Methodology

8.1 Phase 1: Research and Planning

- Study Assembly Language syntax and processor instructions.
- Learn BIOS interrupts and memory management techniques.
- Design the game flow and overall architecture.
- Plan obstacle movement and collision logic.

8.2 Phase 2: Core Development

- Implement screen rendering system.
- Add keyboard input controls.
- Develop player movement functionality.
- Create delay loops for timing control.

8.3 Phase 3: Game Mechanics

- Implement obstacle generation system.
- Add collision detection algorithms.
- Develop score calculation system.
- Increase game difficulty dynamically.

8.4 Phase 4: Testing and Optimization

- Debug logical and runtime errors.
- Optimize loops and instructions for better performance.
- Improve responsiveness and gameplay experience.
- Perform final testing and validation.

9 Expected Outcomes

The expected outcomes of the project include:

- Development of a complete playable car racing game.
- Better understanding of low-level programming concepts.
- Practical experience with hardware-level interaction.
- Improved knowledge of interrupts and memory management.
- Enhanced debugging and optimization skills.
- Understanding of real-time application development.

10 Innovation

This project is unique because it is entirely developed using Assembly Language without relying on modern game engines, graphical frameworks, or external libraries. The project demonstrates direct hardware interaction and manual implementation of system-level functionalities.

The game may also include an advanced extension where obstacle movement patterns simulate basic artificial intelligence behavior, making gameplay more dynamic and challenging.

11 Future Enhancements

The following improvements may be added in future versions of the project:

- Add sound effects using speaker interrupts.
- Introduce multiple difficulty levels.
- Add graphical menus and restart functionality.
- Upgrade to advanced pixel-based graphics mode.
- Implement multiplayer support.
- Add different vehicle types and obstacle variations.
- Store high scores permanently using file handling.

12 Conclusion

This project demonstrates how a real-time interactive application can be developed using low-level programming techniques. By building a complete car racing game in 8086 Assembly Language, the project provides practical understanding of memory handling, interrupt processing, CPU instructions, and hardware interaction.

The project also improves problem-solving abilities, debugging skills, and knowledge of system-level programming concepts. The final implementation serves as a valuable learning experience in understanding how software operates at the hardware level without relying on modern high-level development tools.